Os limites das linguagens regulares

Encontro 7 | 9 de setembro

Linguagens Formais e Compiladores

1
O que é uma linguagem regular

Vocês já viram três notações

autômato finito	recebe a palavra e responde sim ou não
expressão regular	descreve o formato das palavras
gramática regular	produz as palavras, uma regra por vez

Vocês já viram três notações

autômato finito	recebe a palavra e responde sim ou não
expressão regular	descreve o formato das palavras
gramática regular	produz as palavras, uma regra por vez

As três descrevem exatamente o mesmo conjunto de linguagens.

Vocês já viram três notações

autômato finito	recebe a palavra e responde sim ou não
expressão regular	descreve o formato das palavras
gramática regular	produz as palavras, uma regra por vez

As três descrevem exatamente o mesmo conjunto de linguagens.

Esse conjunto tem nome: são as **linguagens regulares**.

O que as três têm em comum

Memória finita.

O que as três têm em comum

Memória finita.

O autômato só se lembra de uma coisa: **em que estado está**.
E o número de estados é fixo, decidido antes de rodar.

O que as três têm em comum

Memória finita.

O autômato só se lembra de uma coisa: **em que estado está**.
E o número de estados é fixo, decidido antes de rodar.

Expressão regular e gramática regular viram autômato.
Então herdam a mesma limitação.

Uma definição prática

Em vez de decorar a definição formal, use esta:

Linguagem regular é aquela que se reconhece com uma quantidade fixa de memória.

Uma definição prática

Em vez de decorar a definição formal, use esta:

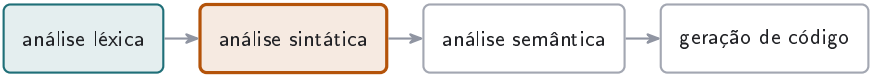
Linguagem regular é aquela que se reconhece com uma quantidade fixa de memória.

A aula de hoje é sobre o que essa restrição impede.

2

Por que isso importa agora

Onde vocês estão no compilador



Onde vocês estão no compilador



A análise léxica vocês fizeram na E1, com autômatos. Funcionou.

Onde vocês estão no compilador



A análise léxica vocês fizeram na E1, com autômatos. Funcionou.

Pergunta natural: por que não usar autômato na próxima fase também?

Olhem este programa

```
se (x > 0) {
  enquanto (y < 10) {
    se (z == 1) {
      faz(a);
    }
  }
}
```

Olhem este programa

```
se (x > 0) {
  enquanto (y < 10) {
    se (z == 1) {
      faz(a);
    }
  }
}
```

Três níveis de chaves aninhadas. E dá para escrever com dez, com cem.

Olhem este programa

```
se (x > 0) {
  enquanto (y < 10) {
    se (z == 1) {
      faz(a);
    }
  }
}
```

Três níveis de chaves aninhadas. E dá para escrever com dez, com cem.

Para saber se as chaves fecham direito,
é preciso lembrar quantas foram abertas.

3

Prática: validando na mão

Antes da teoria, vamos resolver

Escrevam um validador de parênteses. Recebe uma palavra, responde se os parênteses estão balanceados.

Antes da teoria, vamos resolver

Escrevam um validador de parênteses. Recebe uma palavra, responde se os parênteses estão balanceados.

válido	()	(())	()()	(()())	
inválido	(	)	(()	())	

Antes da teoria, vamos resolver

Escrevam um validador de parênteses. Recebe uma palavra, responde se os parênteses estão balanceados.

válido	`()`	`(())`	`()()`	`(()())`	
inválido	`(`	`)`	`(()`	`())`	

Cinco minutos. Não precisa ser bonito.

A solução que todo mundo escreve

```
def valida(s):
    c = 0
    for ch in s:
        if ch == '(':
            c = c + 1
        elif ch == ')':
            c = c - 1
            if c < 0: return False
    return c == 0
```

A solução que todo mundo escreve

```
def valida(s):
    c = 0
    for ch in s:
        if ch == '(':
            c = c + 1
        elif ch == ')':
            c = c - 1
            if c < 0: return False
    return c == 0
```

Um contador. Sobe no abre, desce no fecha.
Se ficar negativo, fechou sem abrir. Se sobrar, faltou fechar.

Funciona

entrada	resultado	contador chegou a
`()`	válido	1
`(((())))`	válido	3
`(()`	inválido	sobrou 1
`())`	inválido	ficou negativo

Funciona

entrada	resultado	contador chegou a
`()`	válido	1
`(((())))`	válido	3
`(()`	inválido	sobrou 1
`())`	inválido	ficou negativo

Uma variável resolveu. E funciona para qualquer profundidade.

Até onde o contador vai

níveis	contador chega a
3	3
10	10
100	100
1.000	**1.000**

Até onde o contador vai

níveis	contador chega a
3	3
10	10
100	100
1.000	**1.000**

O contador assume qualquer valor. Não tem teto.

Até onde o contador vai

níveis	contador chega a
3	3
10	10
100	100
1.000	**1.000**

O contador assume qualquer valor. Não tem teto.

Um autômato tem número *fixo* de estados.
Ele não tem onde guardar esse contador.

A descoberta

Vocês acabaram de resolver o problema usando algo que um autômato finito não possui.

A descoberta

Vocês acabaram de resolver o problema usando algo que um autômato finito não possui.

Uma variável que cresce sem limite.

A descoberta

Vocês acabaram de resolver o problema usando algo que um autômato finito não possui.

Uma variável que cresce sem limite.

Isso é a diferença entre análise léxica e análise sintática.

4

O contador não basta

Agora com dois tipos de delimitador

Parênteses e colchetes juntos. Quais destes são válidos?

```
([])      ([)]
```

Agora com dois tipos de delimitador

Parênteses e colchetes juntos. Quais destes são válidos?

```
([])        ([)]
```

O primeiro está certo. O segundo está errado:
o colchete abriu dentro do parêntese e fechou fora dele.

O que o contador responde

entrada	contador diz	o certo é	
`([])`	válido	válido	
`([)]`	válido	inválido	errou
`{[()]}`	válido	válido	
`{[(])}`	válido	inválido	errou

O que o contador responde

entrada	contador diz	o certo é	
`([])`	válido	válido	
`([)]`	válido	inválido	errou
`{[()]}`	válido	válido	
`{[(])}`	válido	inválido	errou

O contador conta três abertos e três fechados, e aprova.

O que o contador responde

entrada	contador diz	o certo é	
`([])`	válido	válido	
`([)]`	válido	inválido	errou
`{[()]}`	válido	válido	
`{[(])}`	válido	inválido	errou

O contador conta três abertos e três fechados, e aprova.

Ele sabe *quantos* estão abertos. Não sabe *quais*.

A correção: uma pilha

```
def valida(s):
    par = {')':'(', ']':'[', '}':'{'}
    pilha = []
    for ch in s:
        if ch in '([{':
            pilha.append(ch)
        elif ch in ')]}':
            if not pilha: return False
            if pilha[-1] != par[ch]: return False
            pilha.pop()
    return len(pilha) == 0
```

A correção: uma pilha

```
def valida(s):
    par = {')':'(', ']':'[', '}':'{'}
    pilha = []
    for ch in s:
        if ch in '([{':
            pilha.append(ch)
        elif ch in ')]}':
            if not pilha: return False
            if pilha[-1] != par[ch]: return False
            pilha.pop()
    return len(pilha) == 0
```

Guarda **o que** abriu, e confere se o fecha corresponde ao último.

Por que a pilha resolve

A regra do aninhamento é: *o último que abriu é o primeiro que fecha*.

Por que a pilha resolve

A regra do aninhamento é: *o último que abriu é o primeiro que fecha*.

E pilha é exatamente a estrutura com essa regra.

Por que a pilha resolve

A regra do aninhamento é: *o último que abriu é o primeiro que fecha*.

E pilha é exatamente a estrutura com essa regra.

O contador era uma pilha degenerada: guardava só a *altura*,
e jogava fora a informação de quem estava lá.

O que vocês construíram hoje

Um analisador sintático, na sua forma mais simples.

O que vocês construíram hoje

Um analisador sintático, na sua forma mais simples.

Ele verifica se a estrutura aninhada está bem formada.
É a primeira coisa que qualquer analisador sintático faz.

O que vocês construíram hoje

Um analisador sintático, na sua forma mais simples.

Ele verifica se a estrutura aninhada está bem formada.
É a primeira coisa que qualquer analisador sintático faz.

E ele usa uma pilha, que é o que dá nome à máquina
do encontro 10: o **autômato de pilha**.

5
A máquina que faltava

Vamos formalizar o que vocês fizeram

O validador tinha duas coisas: o código que lia os símbolos,
e a pilha que guardava o que estava aberto.

Vamos formalizar o que vocês fizeram

O validador tinha duas coisas: o código que lia os símbolos,
e a pilha que guardava o que estava aberto.

Isso é um autômato finito + uma pilha.

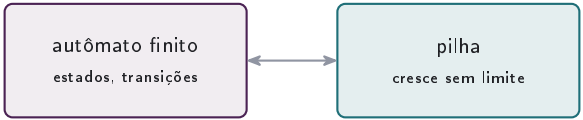
Vamos formalizar o que vocês fizeram

O validador tinha duas coisas: o código que lia os símbolos,
e a pilha que guardava o que estava aberto.

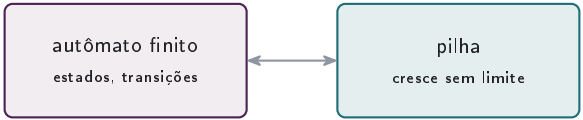
Isso é um autômato finito + uma pilha.

A máquina tem nome: **autômato de pilha**.

As duas partes

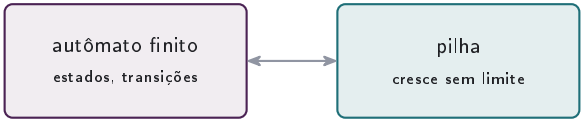


As duas partes



O autômato decide o que fazer. A pilha guarda o que ele precisa lembrar.

As duas partes



O autômato decide o que fazer. A pilha guarda o que ele precisa lembrar.

A memória finita continua lá, nos estados. O que mudou foi ganhar uma memória *extra*, que não tem teto.

Três operações, e nada mais

empurrar	põe um símbolo no topo da pilha
tirar	remove o símbolo do topo
olhar	vê o que está no topo, sem remover

Três operações, e nada mais

empurrar	põe um símbolo no topo da pilha
tirar	remove o símbolo do topo
olhar	vê o que está no topo, sem remover

Só o topo é acessível. O que está embaixo fica inacessível até o de cima sair.

Três operações, e nada mais

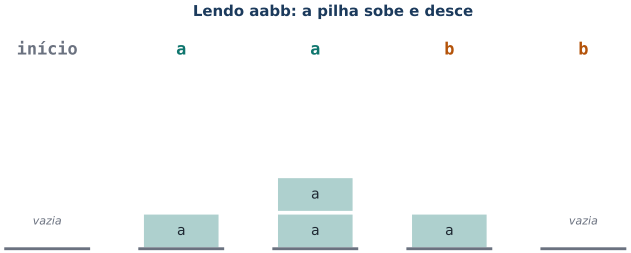
empurrar	põe um símbolo no topo da pilha
tirar	remove o símbolo do topo
olhar	vê o que está no topo, sem remover

Só o topo é acessível. O que está embaixo fica inacessível até o de cima sair.

É a mesma regra do `append` e do `pop` que vocês usaram.

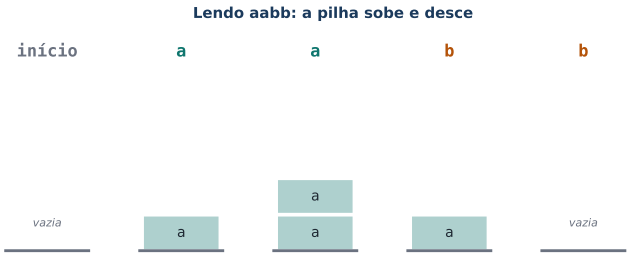
Reconhecendo `aabb`

Cada `a` empurra. Cada `b` tira. Aceita se a pilha ficar vazia no fim.



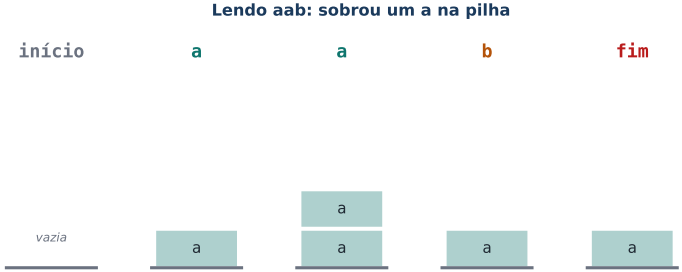
Reconhecendo `aabb`

Cada `a` empurra. Cada `b` tira. Aceita se a pilha ficar vazia no fim.

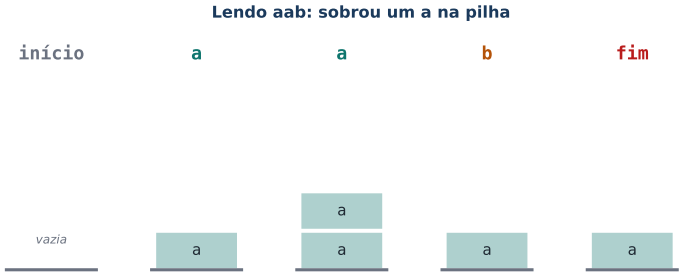


Terminou vazia. Aceita.

E aab

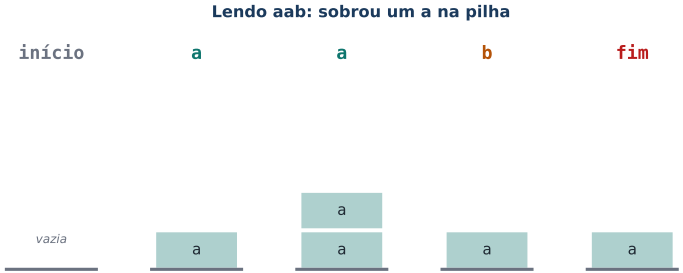


E aab



Dois a empurraram, um b tirou. Sobrou um na pilha.

E aab



Dois a empurraram, um b tirou. Sobrou um na pilha.

Pilha não vazia no fim. Rejeita.

Onde está o ganho

O autômato finito precisava de um estado por cada quantidade possível de a pendentes: a_1, a_2, a_3, e assim sem fim.

Onde está o ganho

O autômato finito precisava de um estado por cada quantidade possível de a pendentes: a_1, a_2, a_3, e assim sem fim.

O autômato de pilha usa *um* estado, e a contagem fica na pilha.

Onde está o ganho

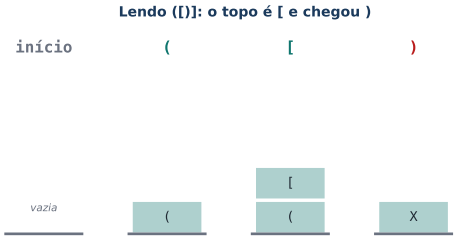
O autômato finito precisava de um estado por cada quantidade possível de a pendentes: a_1, a_2, a_3, e assim sem fim.

O autômato de pilha usa *um* estado, e a contagem fica na pilha.

Estados param de crescer. A pilha cresce por eles.

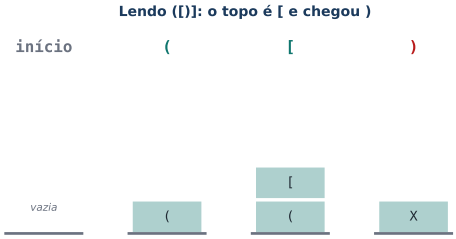
Com dois tipos de delimitador

Agora a pilha guarda *qual* símbolo abriu, não só quantos.



Com dois tipos de delimitador

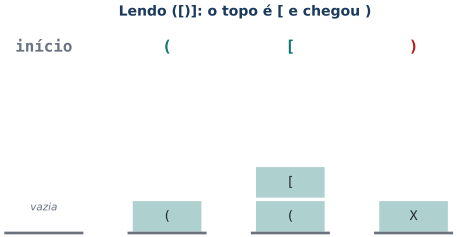
Agora a pilha guarda *qual* símbolo abriu, não só quantos.



Chegou um), mas o topo da pilha é [. Não corresponde.

Com dois tipos de delimitador

Agora a pilha guarda *qual* símbolo abriu, não só quantos.



Chegou um), mas o topo da pilha é [. Não corresponde.

Rejeita. E era exatamente o caso que o contador aprovava.

O que a pilha faz que o contador não fazia

	guarda	**resolve**
contador	quantos abriram	um tipo de delimitador
pilha	quais abriram, e em que ordem	vários tipos

O que a pilha faz que o contador não fazia

	guarda	**resolve**
contador	quantos abriram	um tipo de delimitador
pilha	quais abriram, e em que ordem	vários tipos

E é por isso que compilador de verdade usa pilha, não contador.

Onde isso se encaixa

tipo	linguagens	máquina	memória
3	regulares	autômato finito	fixa
2	livres de contexto	autômato de pilha	uma pilha
0	recursivamente enumeráveis	máquina de Turing	ilimitada

Onde isso se encaixa

tipo	linguagens	máquina	memória
3	regulares	autômato finito	fixa
2	livres de contexto	autômato de pilha	uma pilha
0	recursivamente enumeráveis	máquina de Turing	ilimitada

Cada linha ganha uma memória mais poderosa, e com ela consegue reconhecer mais linguagens.

Onde isso se encaixa

tipo	linguagens	máquina	memória
3	regulares	autômato finito	fixa
2	livres de contexto	autômato de pilha	uma pilha
0	recursivamente enumeráveis	máquina de Turing	ilimitada

Cada linha ganha uma memória mais poderosa, e com ela consegue reconhecer mais linguagens.

Vocês passaram do tipo 3 para o tipo 2 hoje, escrevendo código.

O que vem no encontro 10

gramáticas livres de contexto	a notação para descrever a sintaxe
autômato de pilha formalizado	a máquina que a reconhece
árvore sintática	o que o analisador constrói

O que vem no encontro 10

gramáticas livres de contexto	a notação para descrever a sintaxe
autômato de pilha formalizado	a máquina que a reconhece
árvore sintática	o que o analisador constrói

E a E2 é escrever a gramática de uma linguagem pequena.

O que vem no encontro 10

gramáticas livres de contexto	a notação para descrever a sintaxe
autômato de pilha formalizado	a máquina que a reconhece
árvore sintática	o que o analisador constrói

E a E2 é escrever a gramática de uma linguagem pequena.

Quem entendeu a pilha de hoje vai chegar lá com meio caminho andado.

O que fica

- Linguagem regular é a que se reconhece com memória finita.

O que fica

- Linguagem regular é a que se reconhece com memória finita.
- Autômato, expressão regular e gramática regular são a mesma classe.

O que fica

- Linguagem regular é a que se reconhece com memória finita.
- Autômato, expressão regular e gramática regular são a mesma classe.
- Aninhamento precisa de memória que cresce, e escapa dessa classe.

O que fica

- Linguagem regular é a que se reconhece com memória finita.
- Autômato, expressão regular e gramática regular são a mesma classe.
- Aninhamento precisa de memória que cresce, e escapa dessa classe.
- Um contador resolve um tipo de delimitador. Uma pilha resolve vários.

O que fica

- Linguagem regular é a que se reconhece com memória finita.
- Autômato, expressão regular e gramática regular são a mesma classe.
- Aninhamento precisa de memória que cresce, e escapa dessa classe.
- Um contador resolve um tipo de delimitador. Uma pilha resolve vários.
- Autômato finito mais pilha é o autômato de pilha.

O que fica

- Linguagem regular é a que se reconhece com memória finita.
- Autômato, expressão regular e gramática regular são a mesma classe.
- Aninhamento precisa de memória que cresce, e escapa dessa classe.
- Um contador resolve um tipo de delimitador. Uma pilha resolve vários.
- Autômato finito mais pilha é o autômato de pilha.
- Ele usa um estado só, e deixa a contagem na pilha.

O que fica

- Linguagem regular é a que se reconhece com memória finita.
- Autômato, expressão regular e gramática regular são a mesma classe.
- Aninhamento precisa de memória que cresce, e escapa dessa classe.
- Um contador resolve um tipo de delimitador. Uma pilha resolve vários.
- Autômato finito mais pilha é o autômato de pilha.
- Ele usa um estado só, e deixa a contagem na pilha.
- E é a máquina da análise sintática.

O que fica

- Linguagem regular é a que se reconhece com memória finita.
- Autômato, expressão regular e gramática regular são a mesma classe.
- Aninhamento precisa de memória que cresce, e escapa dessa classe.
- Um contador resolve um tipo de delimitador. Uma pilha resolve vários.
- Autômato finito mais pilha é o autômato de pilha.
- Ele usa um estado só, e deixa a contagem na pilha.
- E é a máquina da análise sintática.

Encontro 10: gramáticas livres de contexto e a árvore sintática.